MusicVault — Music Rating and Discovery Platform

CS 157A · Database Systems · Summer 2026 · Final Project Report

1. Cover Page

Project Title: MusicVault — Music Rating and Discovery Platform

Team Members:

Name	Student ID	Email
Andrew Young	014208948	andrew.b.young@sjsu.edu
Mallika Natarajan	016965312	mallika.natarajan@sjsu.edu
Arsen Keneshbekov	018235035	arsen.keneshbekov@sjsu.edu

2. Project Goals and Description

MusicVault is a web-based music catalog and community platform. It lets users discover music, rate albums, write reviews, build personalized album lists, and organize personal collections, while providing detailed information about artists, albums, genres, record labels, release types, and streaming availability.

The real-world problem it addresses: most existing streaming services are built around *listening*, not *cataloging and discussion*. There is no centralized place where listeners can organize their music, record their opinions, and discover new music through community-generated content. MusicVault fills that gap by combining a structured catalog with community features like ratings, reviews, lists, and user following.

Intended users and roles:

- **Registered Users** browse the catalog, search for artists and albums, rate albums, write reviews, create custom lists, manage personal collections and favorites, and follow other users.
- **Administrators** manage the catalog (artists, albums, tracks, genres, streaming services, record labels, release types) and moderate user-submitted reviews.

3. Functional Requirements and Application Architecture

Features implemented

- Users can register, log in, and manage their profiles.
- Users can browse artists, albums, tracks, genres, and record labels.
- Users can search for music by album title, artist, genre, release year, or record label.
- Users can rate albums using a numerical (half-star) rating system.
- Users can write and delete album reviews.
- Users can create personalized album lists.
- Users can add albums to their personal collection, favorites, or wishlist.
- Users can follow other users.
- Users can view album details such as release type, genres, artists, record label, track listing, streaming availability.
- Users can view community ratings and average album scores.
- Users can view which streaming services an album is available on.
- Administrators can add, edit, and remove catalog entries such as artists, albums, tracks, genres, record labels, release types, streaming services.
- Administrators can remove inappropriate reviews or incorrect catalog information.

System architecture

MusicVault follows a three-tier web application architecture - a presentation layer, an application layer, and a database layer. The presentation layer includes the user interface, allowing users to browse the music catalog, search for albums and artists, manage their profiles, write reviews, create album lists, and rate albums. The application layer handles business logic such as user authentication, authorization, validation of user input, CRUD operations, and communication with the database. It processes requests from the client, executes the appropriate SQL queries, and returns formatted results.

The database layer uses MySQL to store all application data. It enforces referential integrity through primary and foreign keys while supporting efficient retrieval through indexing. Complex relationships such as album genres, streaming services, artist credits, ratings, and album lists are represented using normalized relational tables.

4. ER Data Model

The diagram follows the EER notation used in class. Arrowheads point toward the "one" side of each relationship; double lines indicate total participation. The circled **d** below User marks a disjoint specialization into Registered user and Administrator. Track is a weak entity, drawn with a double rectangle and connected to Album through the identifying relationship Contains, with

track_no as its partial key. Underlined attributes are keys, and avg_rating is shown as a derived attribute since it is computed from the Rates relation rather than stored.

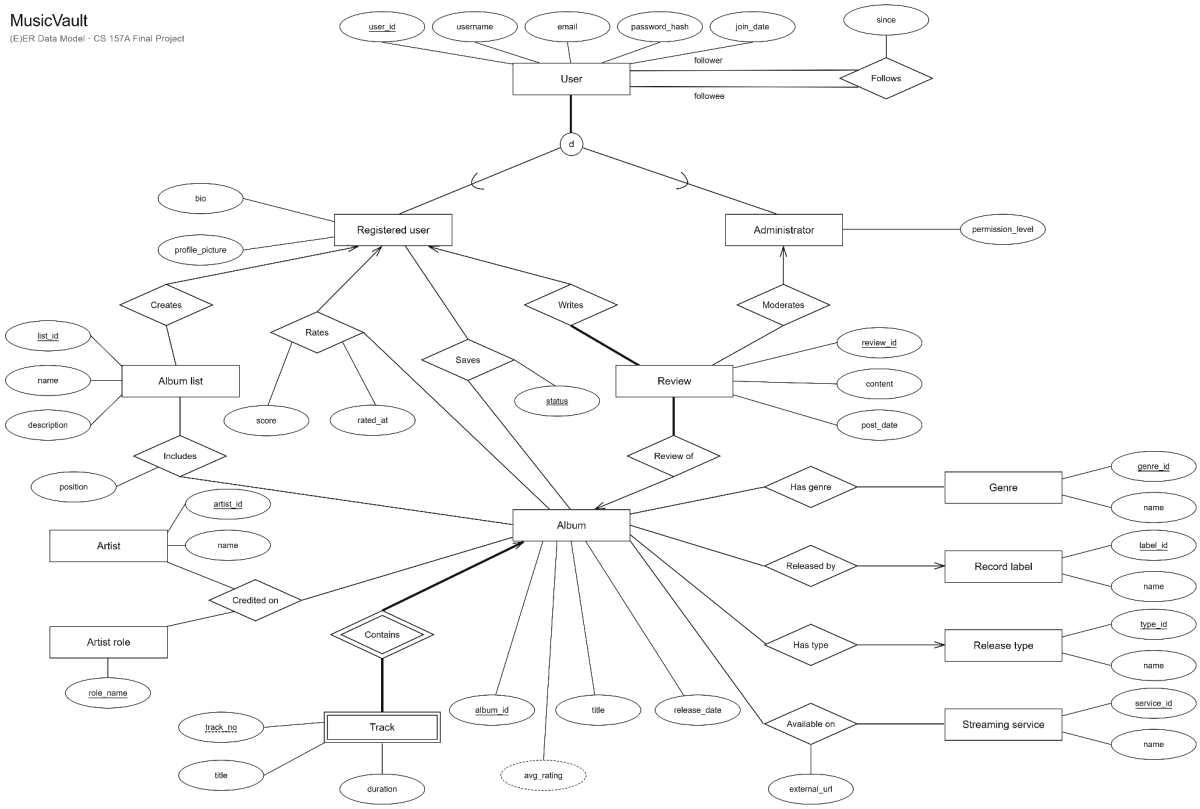


Figure 1. The MusicVault EER diagram.

5. Database Design, Normalization, and Indexing

5.1 Relational Schema

Superclass and subclasses:

- **User**(user_id INT [PK], username VARCHAR(100) UNIQUE NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, join_date DATE NOT NULL)
- **RegisteredUser**(user_id INT [PK, FK→User], bio VARCHAR(1000), profile_picture VARCHAR(255))
- **Administrator**(user_id INT [PK, FK→User], permission_level ENUM('Moderator','SuperAdmin') NOT NULL)

Catalog and lookup entities:

- **RecordLabel**(label_id INT [PK], name VARCHAR(150) NOT NULL)
- **ReleaseType**(type_id INT [PK], name VARCHAR(50) UNIQUE NOT NULL)
- **Genre**(genre_id INT [PK], name VARCHAR(100) UNIQUE NOT NULL)
- **StreamingService**(service_id INT [PK], name VARCHAR(100) UNIQUE NOT NULL)
- **Artist**(artist_id INT [PK], name VARCHAR(200) NOT NULL)
- **ArtistRole**(role_name VARCHAR(50) [PK])
- **Album**(album_id INT [PK], title VARCHAR(300) NOT NULL, release_date DATE, cover_art VARCHAR(255), label_id INT [FK→RecordLabel, NULL], type_id INT [FK→ReleaseType, NOT NULL])

Weak entity:

- **Track**(album_id INT [PK, FK→Album], track_no INT [PK], title VARCHAR(300) NOT NULL, duration TIME) — PK = (album_id, track_no)

Content entities:

- **Review**(review_id INT [PK], content VARCHAR(2000) NOT NULL, post_date DATE NOT NULL, author_id INT [FK→RegisteredUser, NOT NULL], album_id INT [FK→Album, NOT NULL], moderated_by INT [FK→Administrator, NULL])
- **AlbumList**(list_id INT [PK], name VARCHAR(200) NOT NULL, description VARCHAR(1000), user_id INT [FK→RegisteredUser, NOT NULL])

Junction and relationship tables:

- **Follows**(follower_id INT [PK, FK→User], followee_id INT [PK, FK→User], since DATE) — PK = (follower_id, followee_id)
- **Rates**(user_id INT [PK, FK→RegisteredUser], album_id INT [PK, FK→Album], score DECIMAL(2,1) NOT NULL, rated_at DATETIME NOT NULL) — PK = (user_id, album_id); CHECK enforces half-star values 0.5–5.0
- **Saves**(user_id INT [PK, FK→RegisteredUser], album_id INT [PK, FK→Album], status ENUM('Owned','Favorite','Wishlist') [PK]) — PK = (user_id, album_id, status)
- **ListAlbum**(list_id INT [PK, FK→AlbumList], album_id INT [PK, FK→Album], position INT NOT NULL) — PK = (list_id, album_id); UNIQUE (list_id, position)
- **AlbumGenre**(album_id INT [PK, FK→Album], genre_id INT [PK, FK→Genre]) — PK = (album_id, genre_id)
- **AvailableOn**(album_id INT [PK, FK→Album], service_id INT [PK, FK→StreamingService], external_url VARCHAR(500)) — PK = (album_id, service_id)
- **CreditedOn**(album_id INT [PK, FK→Album], artist_id INT [PK, FK→Artist], role_name VARCHAR(50) [PK, FK→ArtistRole]) — PK = (album_id, artist_id, role_name)

5.2 Normalization to BCNF

The schema was designed and normalized to Boyce-Codd Normal Form to minimize redundancy and maintain data integrity. Each entity represents a single concept, and for every nontrivial functional dependency X → Y, X is a superkey of its relation. Lookup information, genres, release types, streaming services, artist roles, record labels, is stored in separate tables, not duplicated throughout the database.

Many-to-many relationships are represented through junction tables including AlbumGenre, AvailableOn, Rates, Saves, ListAlbum, Follows, and CreditedOn. These tables use composite primary keys, uniquely identifying each relationship and removing redundant data. The Track relation is modeled as a weak entity because tracks can not exist independently of their parent album. Its composite primary key (album_id, track_no) guarantees uniqueness within an album.

The specialization hierarchy for User, RegisteredUser, and Administrator separates role attributes into sub class tables while maintaining common user information in a shared superclass. This avoids any null attributes and preserves normalization. No intentional denormalization was performed. The database prioritizes consistency and maintainability over minor query optimizations.

Each relation was checked against its functional dependencies. In Album, the only nontrivial dependency is album_id → title, release_date, cover_art, label_id, type_id, and album_id is the primary key. Track is keyed on (album_id, track_no) → title, duration, with no dependency on a proper subset of that key. Rates gives (user_id, album_id) → score, rated_at, and Review gives review_id → content, post_date, author_id, album_id, moderated_by. User has three candidate keys — user_id, username, and email, each of which determines all remaining attributes, so all determinants are superkeys. ListAlbum likewise has two candidate keys, (list_id, album_id) and (list_id, position), the latter enforced by a UNIQUE constraint. In every case the determinant is a superkey, so all relations satisfy BCNF.

The lookup tables were factored out for this reason. Had Album stored a label name directly, the dependency label_id → label_name would hold on a non-key attribute, violating BCNF and duplicating the label name across every album on that label. Separating RecordLabel, Genre, ReleaseType, StreamingService, and ArtistRole removes those dependencies.

5.3 Indexing

Indexing was applied deliberately rather than broadly. MySQL's InnoDB engine already indexes every primary key and every column with a UNIQUE constraint, and it automatically creates an index on every foreign-key column. Because of this, columns such as User.username, User.email, and Genre.name (all UNIQUE) and all foreign-key columns are already indexed, so

creating explicit indexes on them would be redundant and would only add write and storage overhead. Recognizing what *not* to index was therefore part of the design.

Six explicit indexes were created for the remaining access patterns. Single-column indexes on Album.title, Artist.name, and RecordLabel.name support the catalog search feature, since these are non-unique text columns that are searched but not otherwise indexed. A composite index on Album(release_date, title) supports browsing and sorting albums by release date, and its leading column also serves release-year range filtering. Two further composite indexes are tied to specific queries: Rates(album_id, score) acts as a covering index for the community-average query (SELECT album_id, AVG(score) ... GROUP BY album_id), allowing MySQL to compute each album's average rating from the index alone without reading the table; and Review(album_id, post_date) supports retrieving an album's reviews in newest-first order. Both of these lead with a foreign-key column, so they also satisfy InnoDB's foreign-key index requirement rather than duplicating it.

Running EXPLAIN on the community-average query confirms this: MySQL selects the Rates(album_id, score) index and reports "Using index" in the Extra column, indicating the aggregate is satisfied from the index without reading table rows.

6. Major Design Decisions

Specialization mapping. User is mapped to a superclass relation plus one relation per subclass, rather than a single table with a role/type column. The subclasses are disjoint and total, but they carry their own attributes (bio/profile_picture; permission_level) *and* participate in distinct relationships (only administrators moderate; only registered users write, rate, save, and create lists). Separate relations let each foreign key reference the correct subclass, such as how a review's author must be a registered user, its moderator an administrator, which a single collapsed table could not express.

Many to many relationships. Every M:N relationship becomes its own junction table keyed on the participating entities, carrying any descriptive attributes: Rates (score, rated_at), Saves (status), ListAlbum (position), AvailableOn (external_url), Follows (since), AlbumGenre, and the ternary CreditedOn. One to many relationships (Released by, Has type, Creates, Writes, Review of, Moderates) are folded into the "many" side as foreign keys rather than getting their own tables.

Ternary credit relationship. Credited on is modeled as a true ternary among Album, Artist, and Artist role, with the primary key spanning all three. This lets one artist hold several roles on the same album (e.g., both primary artist and producer), which a binary Album Artist relationship with a single role attribute could not represent.

Weak entity. Track depends on its owning Album. Its primary key is the owner's key plus the partial key track_no, and it cascades on deletion of the album.

Collection vs. favorites. Saves uses a composite key (user_id, album_id, status) so a user can hold the same album under multiple statuses simultaneously (e.g., owned *and* favorited), rather than being forced into one bucket.

Directional following. Following is directional, so "A follows B" and "B follows A" are legitimately distinct rows and no ordering constraint is applied, unlike a symmetric friendship relation which would also forbid reversed duplicates. The only guard needed is against self-following. This was originally written as CHECK (follower_id ≠ followee_id), but MySQL rejects a CHECK constraint on a column that also carries a cascading foreign-key action (error 3823), so the rule is enforced by a BEFORE INSERT trigger instead.

Where business rules are enforced. Referential integrity, half-star rating bounds, uniqueness, and self-follow prevention are enforced at the database level via foreign keys, CHECK constraints, UNIQUE constraints, and triggers. Authentication, administrative authorization (who may edit the catalog), and input validation are enforced at the **application** level. Administrative catalog management is deliberately *not* modeled as ER relationships, it is an authorization concern handled through the User/Administrator role distinction, not persistent data.

Preloaded data. Lookup tables (Genre, ReleaseType, StreamingService, ArtistRole) are seeded with a controlled vocabulary before the app runs. ArtistRole uses role_name as a natural primary key, appropriate for a small, stable vocabulary.

Trade-offs. Two decisions traded performance for normalization. Album averages are computed on demand rather than stored, which keeps the schema free of a derived attribute that could drift out of sync, at the cost of an aggregate query on every browse page, the covering index on Rates(album_id, score) is what makes that acceptable. Review moderation deletes rows rather than flagging them, which is simpler but discards the moderation history that moderated_by was intended to record. The Saves composite key trades a simpler one status per album model for the flexibility of letting a user mark an album owned and favorited at once.

7. Implementation Summary

MusicVault is implemented as a Java Spring Boot web application backed by a MySQL 8 database. The application follows a layered structure: controllers handle incoming HTTP requests, service classes contain the business logic, and repository classes perform data access. Rather than an object-relational mapping framework, the data layer uses Spring's JdbcTemplate with hand-written SQL, keeping the queries explicit and directly tied to the schema. The user interface is rendered server-side with Thymeleaf templates styled using Bootstrap.

The application connects to MySQL through a JDBC connection configured in application.properties. Because the schema is created and populated by the SQL scripts beforehand, schema generation is disabled in the application (ddl-auto=none) and the app operates against the existing tables. Create, read, update, and delete operations are implemented as parameterized SQL statements in the repository and service classes. For example, submitting a rating performs an insert or update against the Rates table, whose composite primary key ensures each user has at most one rating per album.

Authentication and authorization are handled by Spring Security. Passwords are hashed with bcrypt before storage, and a custom user-details service loads each user at login and grants the administrator role when the account has a corresponding row in the Administrator table. The security configuration restricts all /admin routes to administrators, so catalog-management and moderation pages are inaccessible to regular users. The database design directly supports the application's features: average ratings are computed with an aggregate query backed by a covering index, search is implemented with multitable joins across albums, artists, genres, and labels, and album detail pages assemble tracks, credits, genres, and streaming links from several related tables.

8. Demonstration of Example System Run

This section demonstrates MusicVault in operation, walking through the main user and administrator workflows and showing how each is backed by the database. The final figure shows example SQL queries run directly against MySQL, illustrating how the schema and indexes support the application's features.

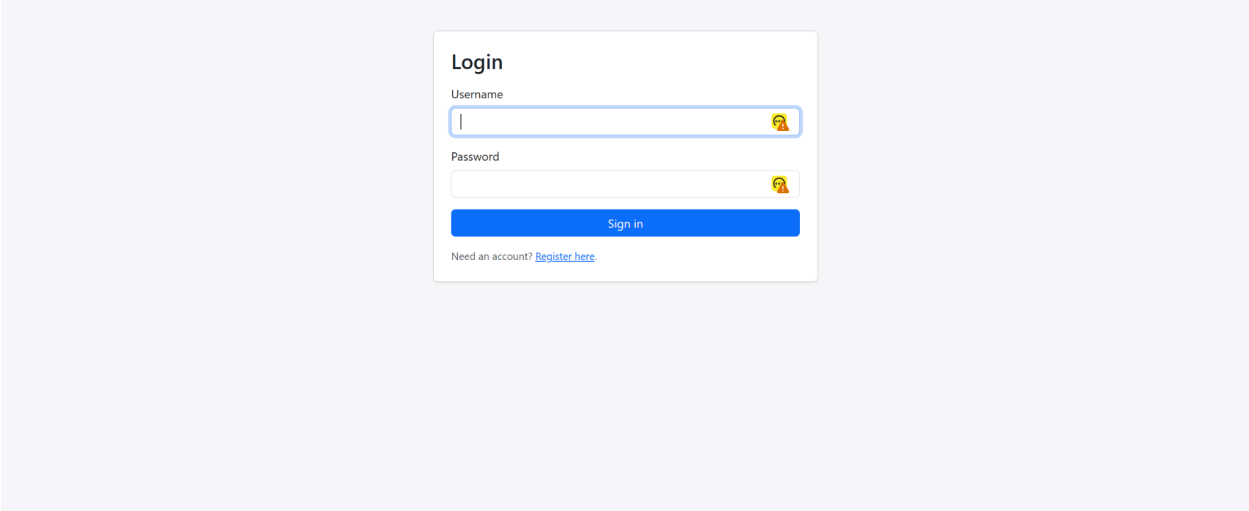


Figure 2. MusicVault’s login page.

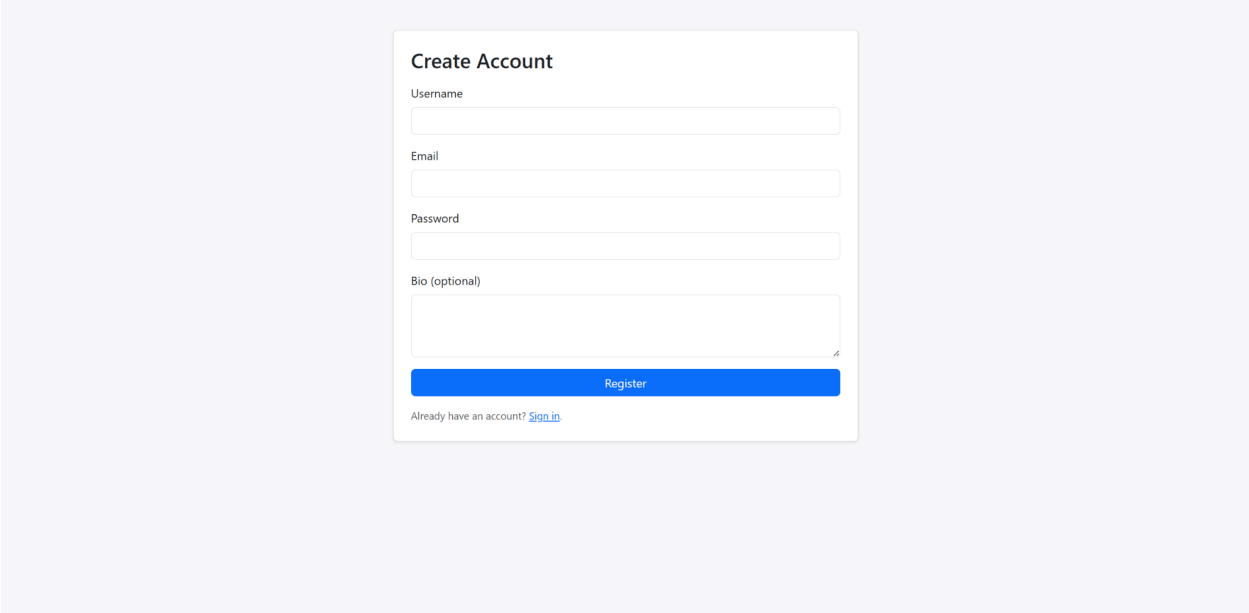


Figure 3. MusicVault's create account page.

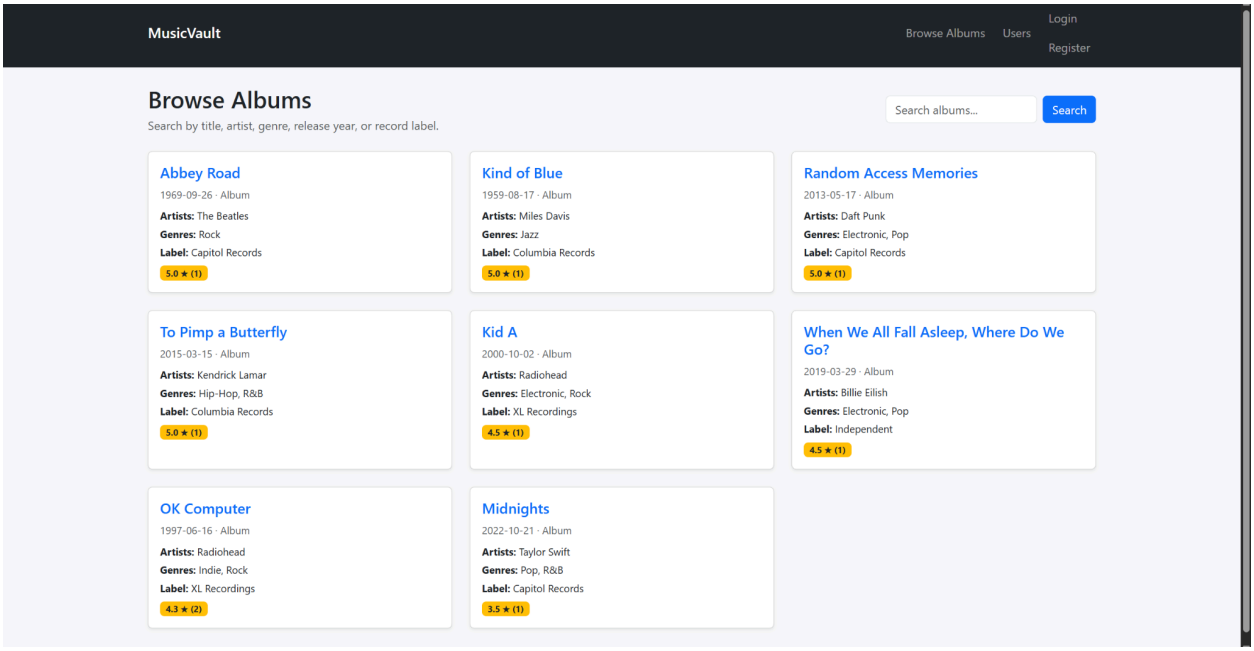


Figure 4. The Browse Albums page. Each album card shows its artists, genres, and label, along with a community rating badge, a live average computed with an aggregate query over the ratings table.

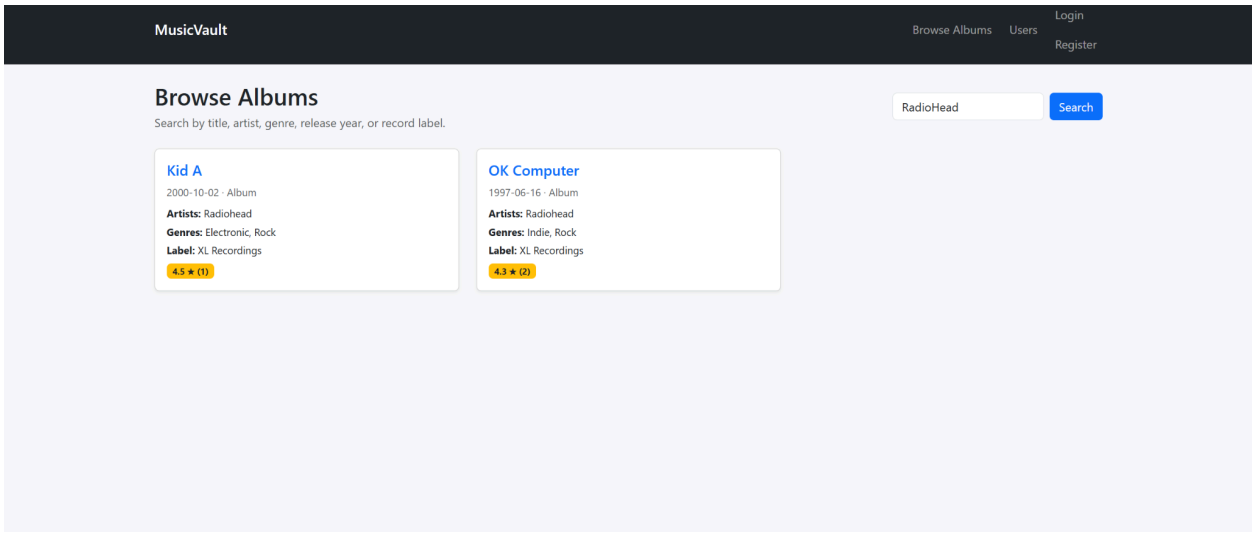


Figure 5. Search results for "Radiohead." Search matches across album title, artist, genre, release year, and record label using a multi table join.

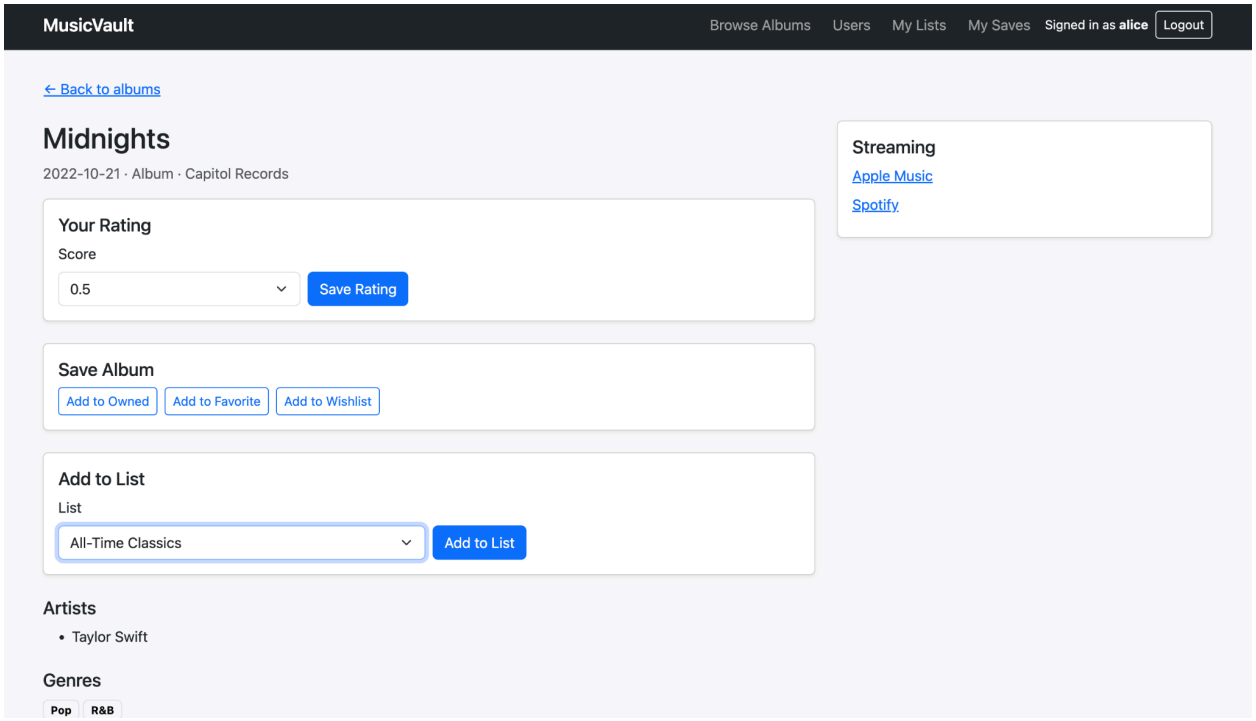


Figure 6. An album detail page. A signed-in user can submit a half star rating, save the album to their collection, favorites, or wishlist, and add it to a personal list. Artists, genres, and streaming links are drawn from related tables.

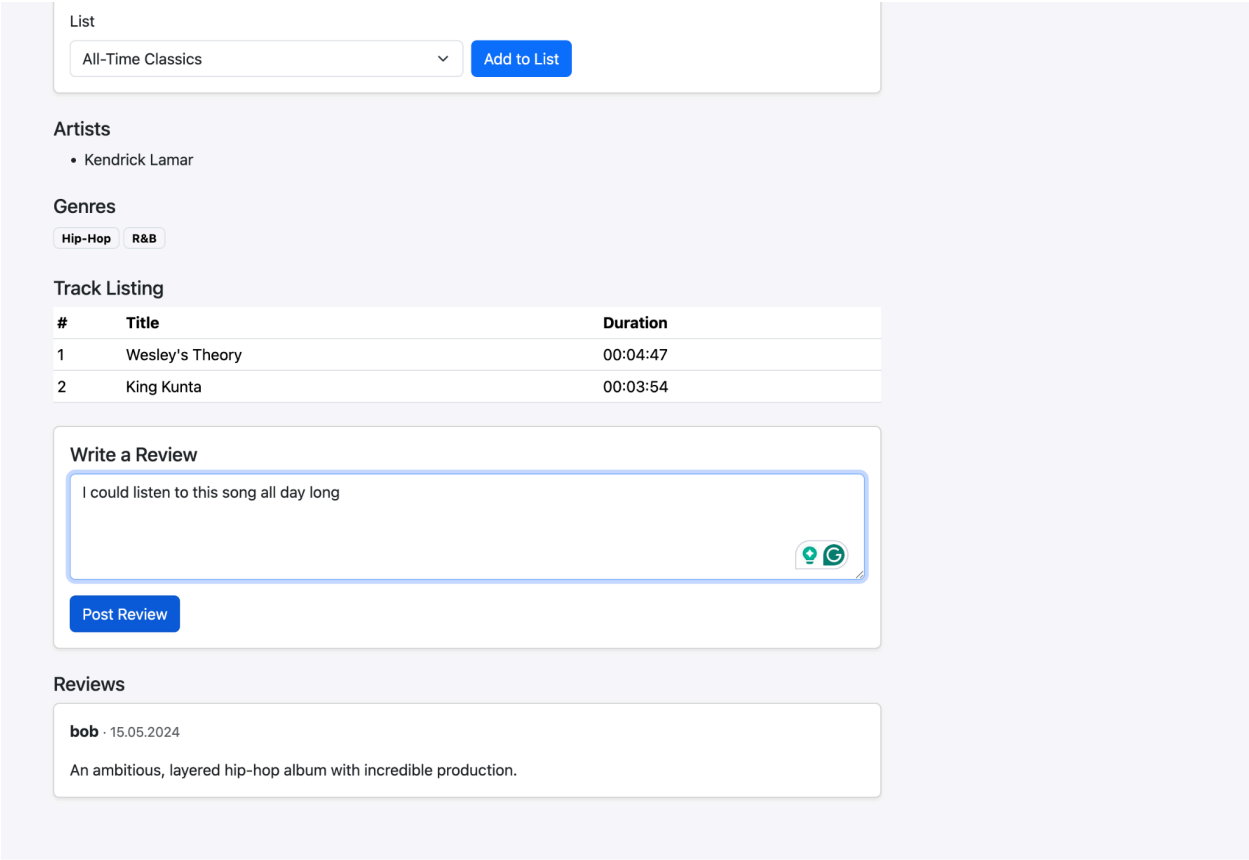


Figure 7. The lower portion of an album detail page, showing the track listing (a weak entity keyed on album and track number) and the review section where users post reviews and read others.

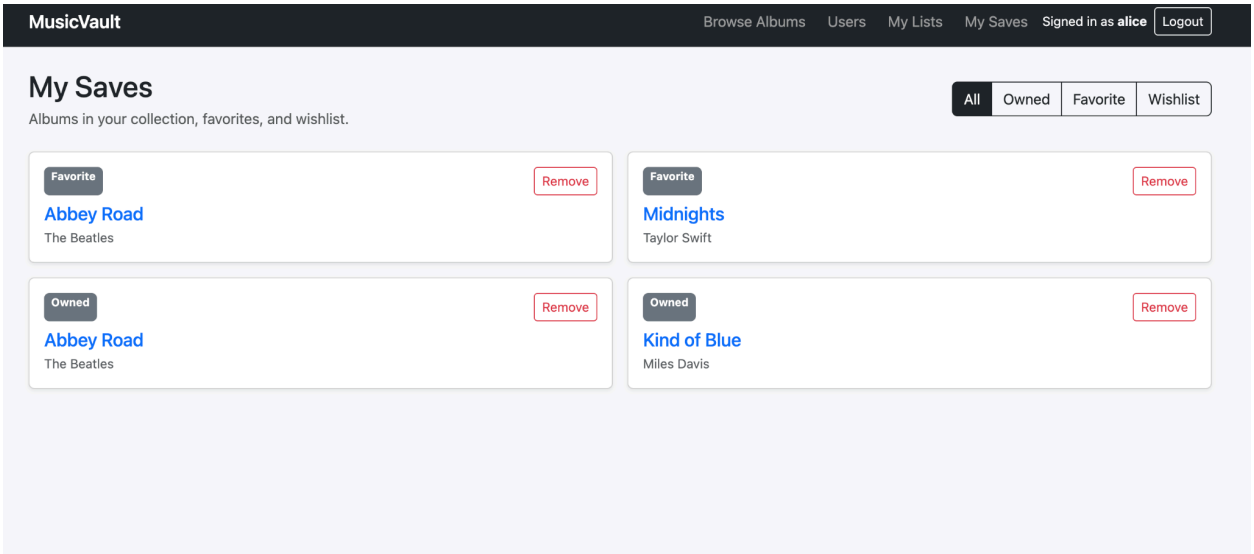


Figure 8. The My Saves page, showing the same user's saved albums filtered by status. Because the Saves relation keys on user, album, and status together, one album can appear under more than one status.

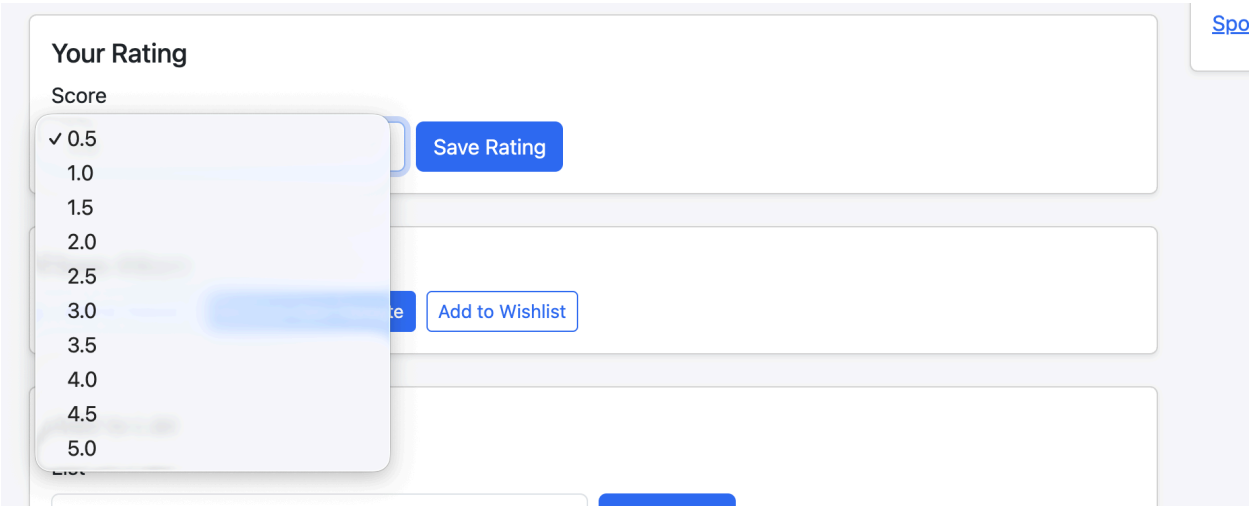


Figure 9. The half-star rating control, offering scores from 0.5 to 5.0 in half-star increments.

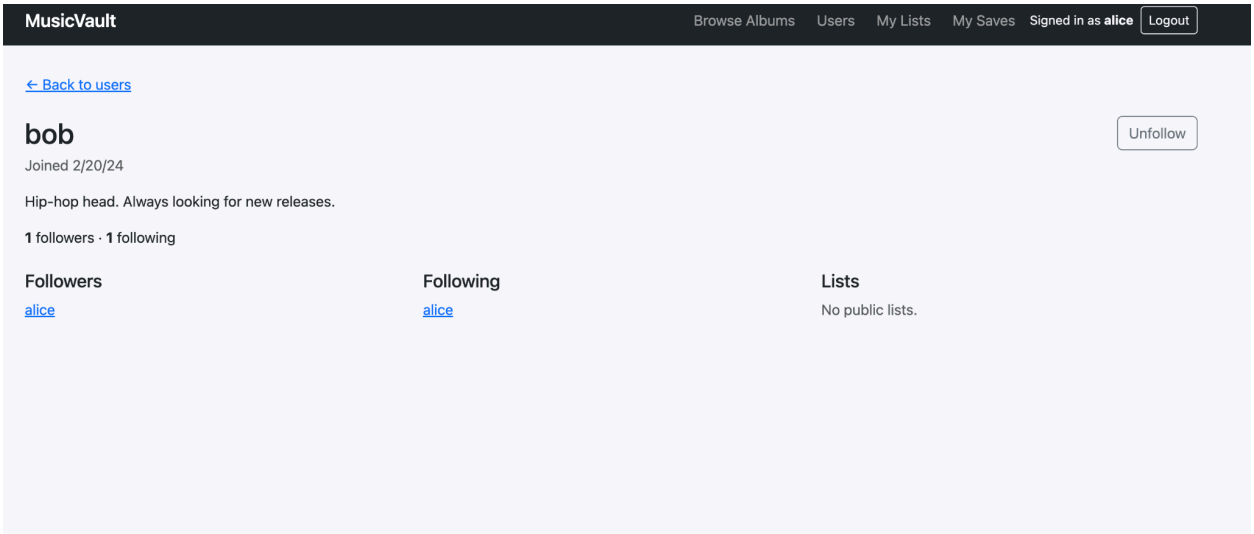


Figure 10. A user profile showing followers and following, populated from the recursive Follows relationship.

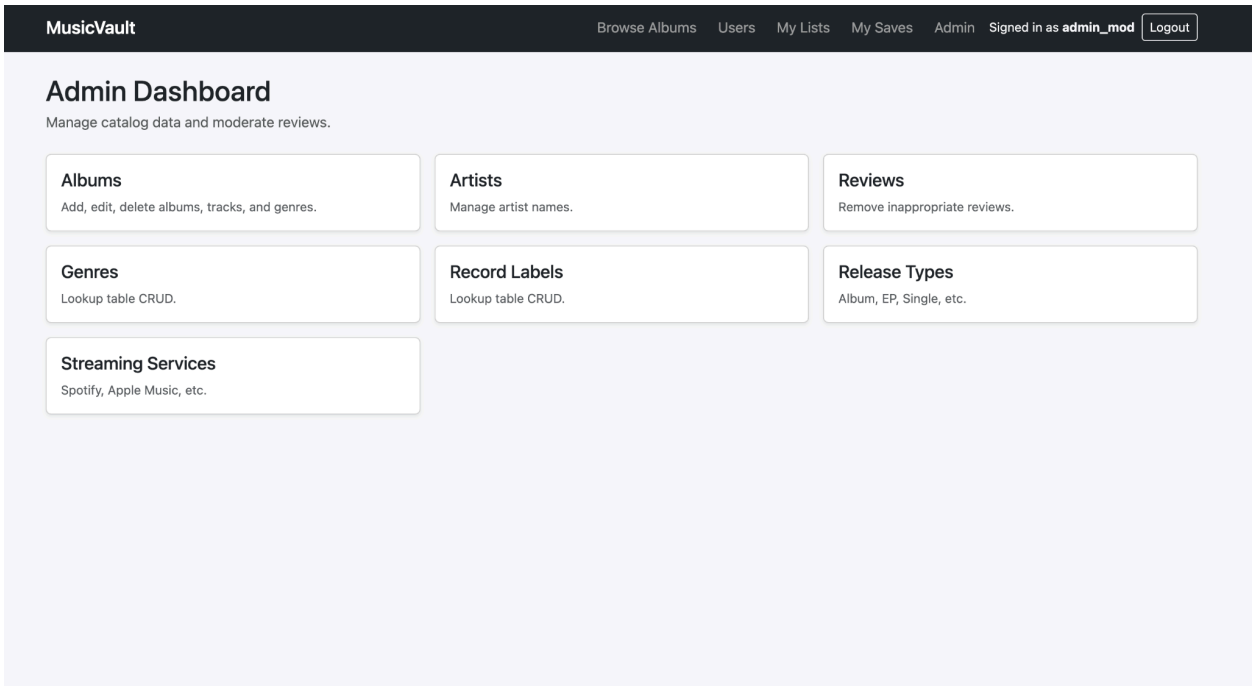


Figure 11. The Admin Dashboard, visible only to administrators. It provides catalog management for albums, artists, genres, labels, release types, and streaming services, plus review moderation.

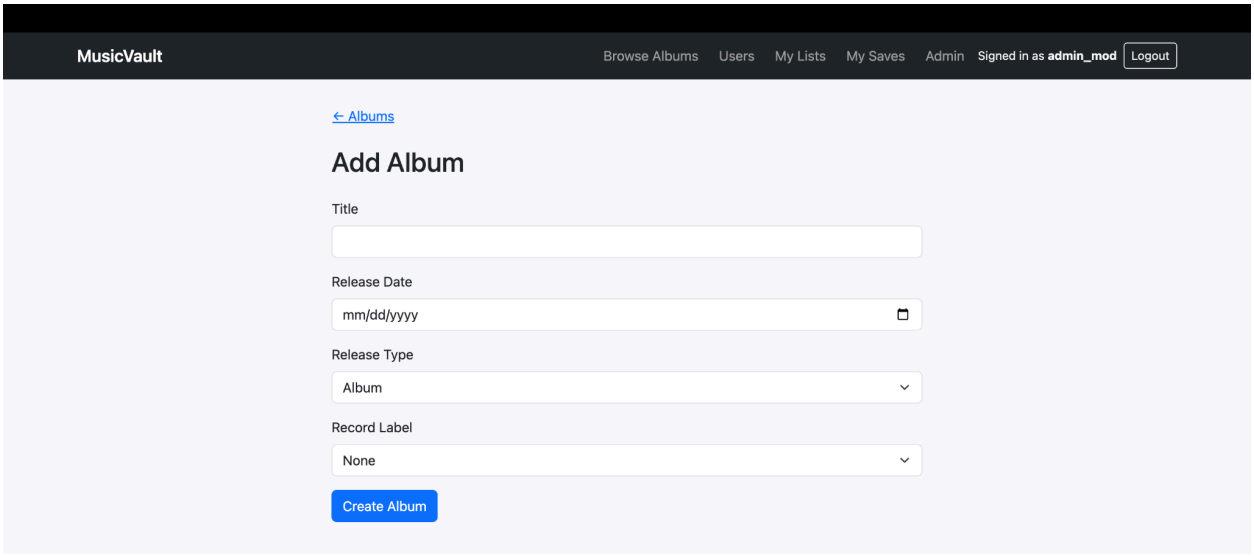


Figure 12. The Add Album form, demonstrating catalog data entry.

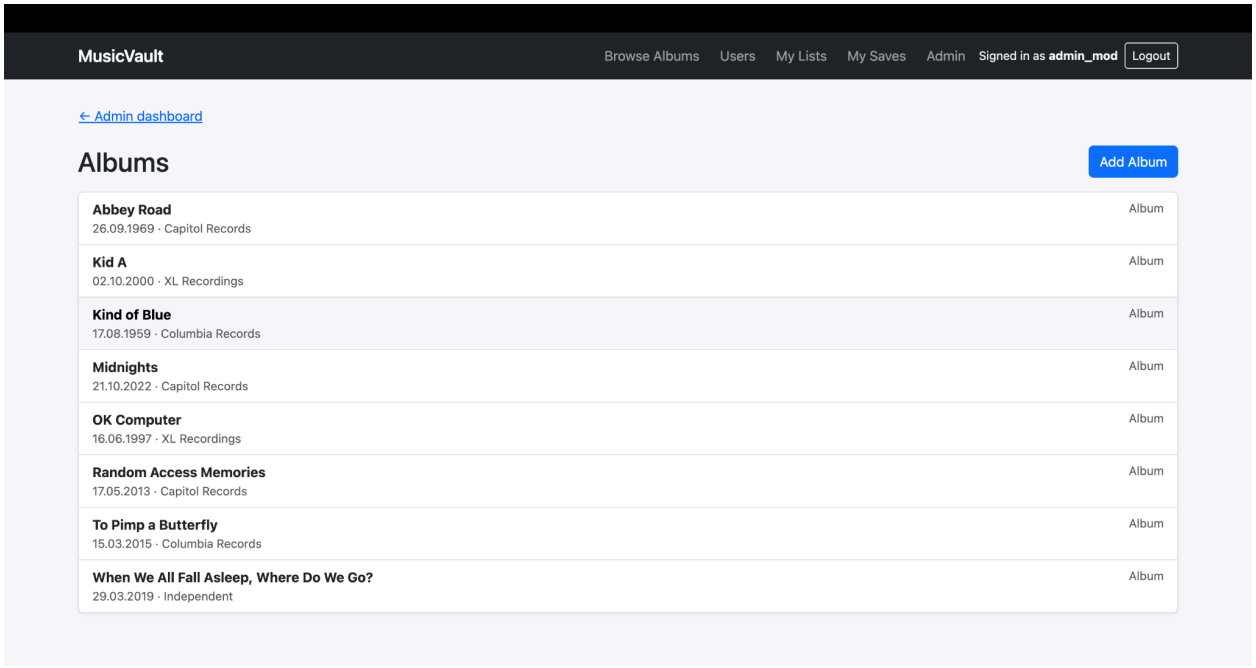


Figure 13. The administrator's album list, from which existing catalog entries can be edited or removed.

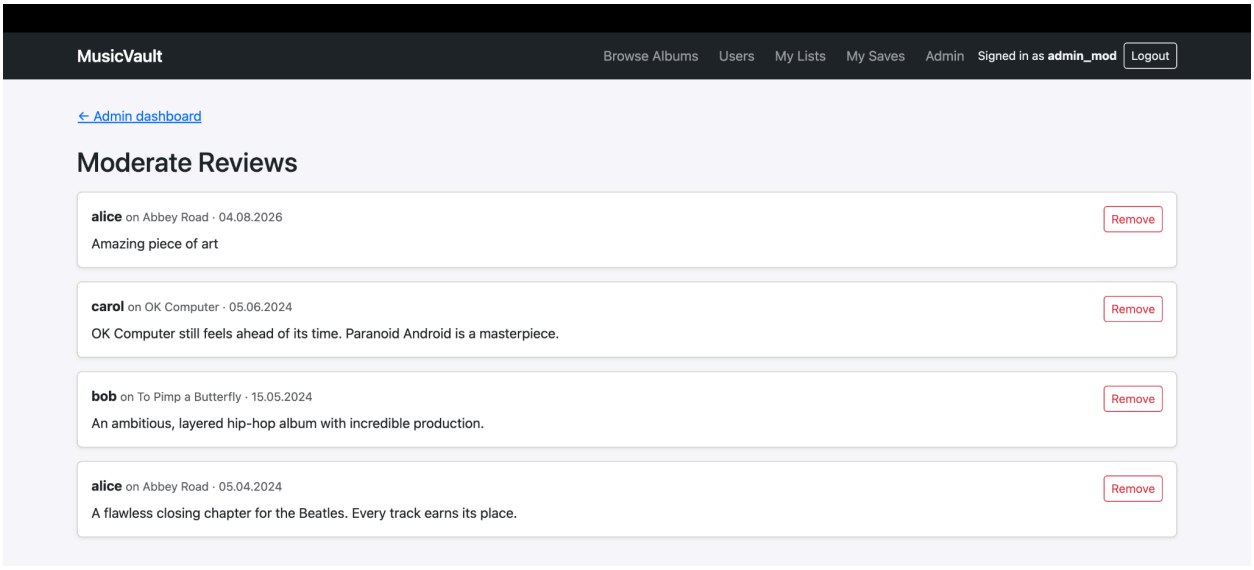
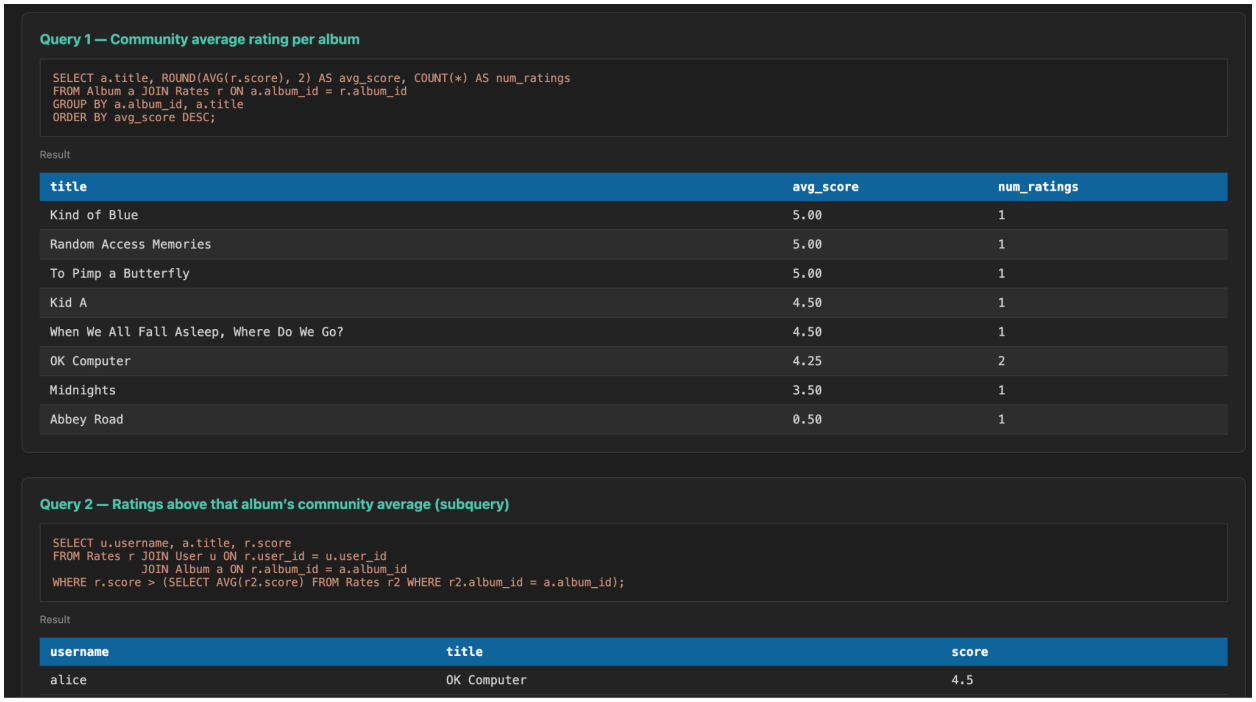


Figure 14. The review-moderation page, where an administrator can remove inappropriate reviews.



Query 1 — Community average rating per album

```
SELECT a.title, ROUND(AVG(r.score), 2) AS avg_score, COUNT(*) AS num_ratings
FROM Album a JOIN Rates r ON a.album_id = r.album_id
GROUP BY a.album_id, a.title
ORDER BY avg_score DESC;
```

Result

title	avg_score	num_ratings
Kind of Blue	5.00	1
Random Access Memories	5.00	1
To Pimp a Butterfly	5.00	1
Kid A	4.50	1
When We All Fall Asleep, Where Do We Go?	4.50	1
OK Computer	4.25	2
Midnights	3.50	1
Abbey Road	0.50	1

Query 2 — Ratings above that album's community average (subquery)

```
SELECT u.username, a.title, r.score
FROM Rates r JOIN User u ON r.user_id = u.user_id
             JOIN Album a ON r.album_id = a.album_id
WHERE r.score > (SELECT AVG(r2.score) FROM Rates r2 WHERE r2.album_id = a.album_id);
```

Result

username	title	score
alice	OK Computer	4.5

Figure 15. Two example queries illustrate how the database supports the application. The first computes each album's community average rating with a join, aggregation, and grouping, the same query behind the rating badges on the browse page. In the results, OK Computer averages 4.25 across its two ratings, while albums with a single high rating appear at the top. The second query uses a correlated subquery to find ratings that exceed their own album's average; it correctly returns only alice's 4.5 rating of OK Computer, since that is the single rating above that album's 4.25 average. Together these demonstrate joins, aggregation, grouping, and subqueries operating over the schema.

Figures were captured at different points during testing, so one album's rating differs between the application screenshots and the query output.

9. Conclusions and Reflections

Building MusicVault reinforced how much of an application's reliability comes from the database design. Translating our EER model into a normalized, BCNF relational schema, and then enforcing business rules, such as referential integrity, half-star rating bounds, uniqueness, and self-follow prevention directly in the database gave us a foundation the application layer could depend on rather than re-implement.

Several challenges were educational as well. We found that MySQL will not allow a CHECK constraint on a column that also carries a cascading foreign key action, which is exactly the situation for our self-follow guard; we resolved it with a BEFORE INSERT trigger, learning a concrete limitation of MySQL's constraint system in the process. We also caught a query that returned an inflated count because a join fanned out over multiple related rows, and fixed it with

COUNT(DISTINCT ...), a reminder to verify aggregate results against known data rather than assuming a query is correct because it runs. On the application side, connecting Spring Security's role-based access control to our User/Administrator specialization required granting the administrator role at login based on the subclass table. Coordinating work across a shared repository, merging each other's contributions, and keeping database credentials out of version control were valuable collaboration lessons.

With more time, we would paginate the browse and search pages so the catalog scales beyond a small dataset, and add a recommendation feature that suggests albums based on a user's ratings and the activity of users they follow. Two improvements follow directly from our design decisions: caching each album's average rating in a trigger maintained column as a documented denormalization for performance at scale, and changing review moderation to hide rather than delete a review so that the moderated_by column keeps a moderation history. The core catalog and community system, however, is complete and fully functional.